Follow Up from Written Notes

Emotet is a modular malware that enters through phishing, establishes persistence, communicates with command-and-control servers, steals data, and delivers additional malware as part of a multi-stage attack. It's not detected not by what it is, but by how it behaves across system and network data over time (its pattern).

Stage 1, Data generated here — email and endpoint data:

- Email headers
- Attachment hashes
- Macro execution events
- Process creation logs (winword.exe → powershell.exe)
- **What the data looks like here: Process logs: winword.exe → powershell.exe → cmd.exe → emotet.exe**
- **Features to extract for my ML — structured log data:**
    - ➔ Parent–child process relationships
    - ➔ Suspicious process chains
    - ➔ Office apps launching scripting tools
    - ➔ Execution timing (immediate vs delayed)

Stage 2, Execution and Persistence – classic host-based data:

- Registry modification logs, and new registry keys
- File system changes
- Scheduled task creation
- Windows Event Logs
- Files written to unusual folders
- **These are strong signals, and normal apps rarely do these quietly. Typical features:**
    - ➔ If something is happening without user interaction, it's suspicious. Emotet adds startup entries without you knowing.
    - ➔ Registry write frequency. Emotet writes registry entries every time it runs to make sure it survives. High frequency is suspicious.
    - ➔ File path entropy (random-looking name). These random names avoid detection and change every infection. A lot of randomness is suspicious.
    - ➔ Persistence mechanism type (task vs registry). How malware makes itself run again after restart. Normally, apps will ask permission and use installers. But installers will do this silently and often use multiple methods to achieve the:
        - ➔ Registry, 'run this program on startup'
        - ➔ Scheduled task. 'Run this every X mins'

Stage 3, Command & Control (C2 Communication), data generated here:

- Outbound network connections
- Suspicious IP/domain traffic
- Beaconing patterns (regular check-ins)
- **Where network-based ML peaks. The typical features are:**
    - ➔ Connection frequency per min
    - ➔ Time interval regularity
    - ➔ Destination IP reputation
    - ➔ Packet size patterns

Stage 4, Data Theft and Reconnaissance, data generated here is mixed host + application-level data

- Credential access logs
- Memory artifacts
- Unusual API calls
- Email scraping behavior

Stage 5, Malware Delivery, where multi-stage attacks happen, happening silently. The data generated here is:

- Secondary payload downloads
- New process trees
- New C2 traffic
- Sudden privilege escalation
- **What the data looks like here:**
    - ➔ New binaries downloaded
    - ➔ Sudden new process trees
    - ➔ New outbound traffic spikes
- Typical features:
    - ➔ File hash novelty — 'how new or unseen is this file'. A hash is a fingerprint, known software has a known hash. But new malware means this hash has never been seen before.
    - ➔ Download → execution time gap. Think of when you normally download a file. What's not normal is for a file to download and it runs immediately. Humans pause, but malware doesn't.
    - ➔ Explosion in process count. This means a sudden increase of running programs, if it suddenly jumps from running 150 processes to 230?? This happens due to malware spawning helpers, or dropping payloads.
    - ➔ Changes in traffic volume. How much data is being sent/received. Malware causes constant background traffic, or sudden large uploads/downloads. This is a strong signal due to unexpected traffic.

Goal of my model: learn the patterns in data. Malware doesn't say it's malware. It only leaves patterns in data, and the goal here is for my model to pick up on those patterns.

1. Execution, Stage 1: Process logs as the data, used for sequences and graphs for ML
2. Persistence, Stage 2: Registry and files for data, used for anomaly detection in the ML model
3. Network, Stage 3: PCAPs and flows for the data, time-series in the model
4. Payload, Stage 4: binary events, for classification in the model.
5. Timeline, Stage 5: all of the above combined, multi-stage modeling.

Analyzing Logs: How Emotet shows up in logs. It's patterns, it's not clearly and bluntly stated malware. Here, we notice that windows launches powershell.exe. But that's not supposed to happen, Word shouldn't launch that. We also notice the same IP, and similar packet size — this is beaconing.

Example: simplified network + process dataset

Time	Host	Process	Parent Process	Destination IP	Bytes Sent	Interval (sec)
10:01	PC-01	chrome.exe	explorer.exe	142.250.x.x	12000	—
10:02	PC-01	winword.exe	explorer.exe	—	—	—
10:02	PC-01	powershell.exe	winword.exe	185.203.x.x	980	300
10:07	PC-01	powershell.exe	winword.exe	185.203.x.x	970	300
10:12	PC-01	powershell.exe	winword.exe	185.203.x.x	960	300

rundll32.exe — one of Emotet's strongest signatures. This is commonly used by Emotet to load malware.

- Word → PowerShell → rundll32

```
makefile

Time: 10:15:12
SrcIP: 10.0.5.23
DstIP: 185.203.116.45
DstPort: 443
BytesSent: 984

Time: 10:20:12
SrcIP: 10.0.5.23
DstIP: 185.203.116.45
DstPort: 443
BytesSent: 978

Time: 10:25:12
SrcIP: 10.0.5.23
DstIP: 185.203.116.45
DstPort: 443
BytesSent: 982
```

Abnormal Patterns:

- Random IPs. This isn't Google nor Microsoft.
- Every five minutes, humans don't browse at this pattern.
- Similar packet size, this is an automation pattern.

This is beaconing… Emotet is beaconing to a command-control server. This is what checking in repeatedly looks like.

```
makefile

Time: 10:31:02
Process: powershell.exe
Action: DownloadFile
URL: http://91.214.xxx.xxx/5sd3.exe
SavedAs: C:\Users\j.doe\AppData\Roaming\3kd92.exe

Time: 10:31:04
Process: 3kd92.exe
Action: Execute
```

Secondary payload download.

- Spotted a random file name
- The download to execution gap is so small. Only 2 seconds.
- It executed without user action.

Refresher video https://youtu.be/oYQrIgx0wV4?si=5KEvkLFv_mvgrojW

- Used to steal credentials, sensitive information, and financial data.
- Can also load other malware onto the victim's device to spy on you
- Polymorphic nature: every time it's accessed, its code changes just a bit. This makes it easy to hide.
- Once a victim is infected, it can spread to other devices in the network, and it also uses brute force to crack passwords.
- Mainly spreads through spam, phishing email.
- Used to target the financial sector.

https://www.malware-traffic-analysis.net/index.html

On https://www.malware-traffic-analysis.net/2025/06/13/index.html, I downloaded the pcap zip, and I analyzed the traffic on Wireshark. The first thing I checked for was the repeating IP address.

I went to statistics → conversations → and then IPv4.

I found the IP of the internal machine, aka the victim, where the host talked to many external IPs over time. These two addresses exchanged data over 3 minutes.

- Internal machine address: 10.6.13.133 … this is the victim
- External Server: 142.250.115.106

I've concluded the address of the infected machine from the characteristics of an Emotet attack (first stage to many other attacks):

- The address appears in almost every row
- It's talking to many external IPs, and it's dominating the traffic I'm seeing.
- But this alone is not enough to prove that it's malware, this is just one of the many characteristics of Emotet. There's a mix of both normal and abnormal traffic, I need to spot more beaconing. Beaconing is found easily by filtering.

So I tried to find some sort of beaconing by sorting the duration of this pcap, but there is none to be found. This file doesn't contain clean C2 beaconing, the way I'm familiar with it… there is no periodic C2-style beaconing to be found here, it just looks like normal, safe application and cloud service communication based on this analysis.

I applied the filter: ip.addr == 10.6.13.133 && dns, to spot weird or random domains to detect some sort of sign to confirm whether this is malware or not. This traffic appears to be very normal and safe, it had some noise, but none that was significant or a 'threat'. Some details of the traffic I observed… repeated domains:

- wpad.massfriction.com — normal, used to repeatedly ask if there's a proxy that should be used. It's just noise.
- msftconnecttest.com — windows uses this to check internet connectivity, it's strange NOT to see this.
- More expected traffic for basic MS services, windows updates, logins; login.live.com, windowsupdate.com, bing.com.
- A bunch of CDNs, cloud infrastructure.

Let's say this file was malware, and it was an Emotet-style DNS, there would be certain patterns to fish for:

- Random looking domains
- Domains queried only one single time
- A low TT

In conclusion, this PCAP just helped me get familiar with analyzing traffic, using Wireshark, although I'm familiar with these features and filters on Wireshark, and it showed me normal traffic and PCAP behaviour. I couldn't spot any Emotet C2 behavior, but this could be of use when I come to train my ML model on normal, clean, and safe PCAP. It scales as a good comparison and will come in handy when I begin my feature validation. However, for now, it's of no use for me.

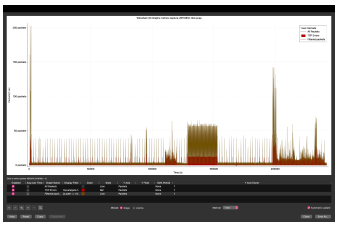
https://www.stratosphereips.org/datasets-ctu13

This traffic exhibits Emotet-like C2 behavior, this is a dataset that's agreed upon and trusted by analysts and many studies related to Emotet. It showcases the behavioral characteristics and nature of an Emotet attack.

- I'm able to acknowledge that these traffic dates back, but it still models Emotet-like C2 behaviour.
- The issue here is that these files are from 2011-15, and the concern here is learning the C2 properties and patterns in order to identify the way Emotet behaves in the datasets.
- We can't really say that this data is necessarily outdated in this case because we're only concerned with patterns, and patterns don't change over the years, the data will change but the patterns that are to be identified will remain the same because they represent the style of attack. These properties include
  - Periodic beaconing
  - Low volume and persistent traffic
  - Encrypted payloads

I selected scenario 3, and the key takeaway from this scenario is: What does Emotet-like command-and-control behavior look like at the network level, and how can it be characterized?

- I downloaded the pcap file and I followed the instructions in the README file.
- <u>The infected host: 147.32.84.165. This is the botnet infected machine that's going to be exploited.</u>
- The normal hosts/IPS, these are not concerning
  - ➔ 147.32.84.170 (amount of bidirectional flows: 217614, Label: Normal-V42-Stribrek)
  - ➔ 147.32.84.134 (amount of bidirectional flows: 1934, Label: Normal-V42-Jist)
  - ➔ 147.32.84.164 (amount of bidirectional flows: 9160, Label: Normal-V42-Grill)
  - ➔ 147.32.87.36 (amount of bidirectional flows: 3686, Label: CVUT-WebServer. This normal host is not so reliable since is a webserver)
  - ➔ 147.32.80.9 (amount of bidirectional flows: 28, Label: CVUT-DNS-Server. This normal host is not so reliable since is a dns server)
  - ➔ 147.32.87.11
- Spotting an interaction like: 147.32.84.165 <-> 95.211.xxx.xxx → tells me that the infected internal host is talking to an external system, and this is where C2 usually lives.
- I then analyzed the I/O graphs, the spikes represented beaconing. This traffic was used to model Emotet-like command and control behavior. These graphs says a lot about the behaviour of this traffic:
  - ➔ Thin spikes represent the low-volume and repeated communication, a key characteristic in Emotet traffic. In some areas, there's sustained activity, as well as bursts of activities.
  - ➔ All of the above represents the persistent C2 check in behavior described earlier. The infected host is 'phoning home', asking for instructions or an order on what to do next, it's also regularly sending status info.



  - ➔ We can also see that it's trying to maintain a presence and attempting to keep the botnet alive which is Emotet behaviour.
  - ➔ Sudden tall spikes: C2 is sending new instructions, the client acknowledges and responds to them, and packets are being exchanged. And since Emotet is a loader, they could also be representing the fetch of additional modules, or updated configs, just executing some instruction it's been given — this causes a momentary surge in traffic, and then it goes back to its quiet beaconing nature. These spikes are NOT how humans browse, because it goes quiet, it spikes, and then it

goes quite again. The botnet here could also be attempting to switch servers due to some sort of failure.

- ➔ The presence of intermittent high-intensity traffic bursts suggests episodic tasking or payload retrieval, consistent with the modular, loader-based design of Emotet-like malware. This is passive beaconing, that later on becomes active command execution (the big block)
- ➔ Large, consistent block of spikes: sustained period of high traffic, but it does drop back to the same beaconing that was occurring before. This block is a successful payload download, it's the transition from an idle bot to an active one, preparing for lateral movement. This is where we see credential harvesting in order to follow the second stage of the malware. It will return back to normal in order to remain present and alive, just normal behaviour, it's alive and waiting for instruction. Once instructed with something, it'll do it, and then it will go back to being quiet and hiding.

https://attack.mitre.org/software/S0367/
Paper: Exploring Emotet's Activities

Document dropper includes five URLs, they're embedded in case of one link failing, there's a bunch of others that can still make the infection attempt a success.

1. URL that hosts the Emotet executables, it leads to the actual program. These sites are usually normal looking, legit websites that are hacked, or temporarily used to store the malware file.
2. URLs that host Emotet documents, and these documents don't look suspicious, they're used to carry out a successful social engineering attempt in order to deliver the executable. It's the initial malicious document, and it includes normal words that would get the average person to click on the document: PAYROLl, DOC, Smallbusiness, Commercial.

- And what makes these two different? Each one targets a different victim. The document URLs target the humans, they click them. And then the executable URLs target the machine, the malware only sees this, they look random.

Good way to summarize it:

1. Email, phishing, get the victim to just open the email
2. The document URL, it looks business-like, innocent and targets the human.
3. Executable URLs, those target the machine, they're random looking.
4. Finally, the Emotet executable runs.

Emotet breaks its own code on purpose and secretly stores the missing pieces somewhere else in memory, and then jumps out to fetch them while running.

- Do not confuse it with packing, obfuscation happens after loading, packing happens before. But Emotet uses both.
- Obfuscation: making the program look broken or incomplete when you analyze it on disk, but it actually runs perfectly fine in memory. At runtime, Emotet is fixing itself quietly, but antivirus, disassemblers, and static scanners see nonsense or missing code.
- Obfuscation is done for a reason, it breaks static analysis, confuses disassemblers, and makes the job of an analyst a lot harder, they'd have to analyze the program/function in a more complex and dynamic manner.

Obfuscation makes Emotet almost impossible to understand while it's just on disk, it'll really only make sense while it's running. Emotet can only be detected by behaviour, the binaries won't help in detecting Emotet because they look different everytime. Analyzing the static state of the program or just on disk won't help at all, the program needs to be running and in action in order to spot Emotet, simply analyzing the code or the binaries won't help. If analyzed statically, you'll see a broken program that somehow executes correctly.

Recall: Emotet is just the loader, this is why it's the first point of analysis in this project, it's like the first entry point to a lot worse. It's just stage one, and its only job is to install other malware, it's just the gateway.

A normal program consists of a call for a function, the instructions inside it, and it returns. But Emotet changes certain things in the structure that we're used to in order to work:

- Removes the first few instructions of a function, making the function look incomplete. This can mean changing the function prologue or setup instructions.

- Emotet will replace normal instructions with a jump to somewhere else (JMP). This way when the function is called, execution immediately jumps away. If we were to look at this program for analysis, we'd naturally assume that the function is broken.
- As for the missing instructions that were removed initially… those get hidden away in a hidden memory area. It's allocated by the loader, and it's completely outside the main executable modules. This area consists of the missing instructions, junk code, and another jump back – Jump Table, this is what this whole hidden area is called.

Emotet Running:

1. Function is called
2. Immediate jump away. So instead of us seeing a normal running function, we see a JMP, the jump table explained above.
3. Jump table now runs the missing instructions, and it'll execute the instructions that were removed at first, doing exactly what the function should have done.
4. The program jumps back to the original function after finishing the missing setup.
5. Function continues normally, nothing looks odd and it'll continue running as expected, and nothing looks strange at runtime.

Dumpling the binaries WILL FAIL:

- If we just dump the modules it will result in a broken executable.
- The jump table is not inside the binary, it only exists after the loader runs and static tools never see the full program.
- From an analyst's perspective, it'll look like missing instructions, weird jumps, broken control flow.

Configuration: Emotet carries a built-in list of servers it should talk to, and the keys needed to encrypt those conversations. This means that Emotet already knows where to call home (IP), and how to connect (through which port).

- It doesn't need DNS to start communicating, and it also doesn't need to discover its servers. Just as soon as Emotet runs, it can start beaconing and carrying out its typical behaviour.
- This is good for Emotet because it can still directly talk to its C2, even if the DNS is blocked or monitored.
- Multiple C2s are stored for the same reason that multiple URLs exist in the document droppers explained previously. These are called fallback servers, in case a C2 is down, it'll try its other options — keeping it alive.

RSA Keys: in the case of Emotet, these RSA keys are embedded in the configuration, and only the attacker has the key. The public key encrypts the data, and the attacker's private key decrypts it.

- The defenders can see the traffic but can't read it, this way the stolen data stays confidential and commands can't be forged by the defender.
- Ties back to the fact that the content of the packets, as analyzed in previous sources, does not matter. With Emotet, it's more important to focus on the patterns of the traffic and its behaviour.

Summary: Emotet embeds its command servers and encryption keys directly in its binary, and communication is done using encrypted web traffic.

- Traffic behaviour looks normal, and it will dynamically load modules (tools) to steal data/send spam. It's a loader for exploitative malware.

Emotet consists of two parallel infrastructures running at the same time. This includes:

- Its own C2 servers
- Its own RSA encryption keys
- Its own deployment schedule

They all drop the same malware, target the same victims, but sometimes operate on alternating days.

- These two infrastructures exist in case one goes down, the other survives and keeps going. Also, it's harder to track and take down. And from the outside, it looks like Emotet is one single thing, but it's actually two separate things.
- And in terms of traffic, Emotet may look similar (timing patterns, burst behaviour, client-server rhythm), but it's actually coming from different IP clusters with different crypto-material.

Different IP Clusters: Emotet has many servers spread all around the world, they change frequently and as mentioned above, it can sometimes have two infrastructures at once.

- One day it'll talk to a certain IP, another the next week, and then a COMPLETELY different IP after.
- So if your detection is IP-reputation based… it's simply not going to work out. The IP changes, but the behaviour remains the same.

Different Crypto Material: Emotet will embed different RSA public keys per infrastructure. This means that the encrypted traffic from botnet A is cryptographically different from botnet B.

- So if an analyst is examining, the packet contents will look completely different, decryption is impossible, and the signatures don't match.
- Patterns and behaviors are the same: the packet sizes are similar, timing is similar, etc… the lock is different, but the movement is the same.

So when building a ML model, it's important to base the detection on certain features like:

- Timing
- Flow statistics
- Packet size distributions
- Connection frequency
- Burst detection
- Sequence modeling

The model needs to detect the pattern, because the pattern is going to remain the same, even if the data is different.

Emotet's activity is identified through consistent behavioral artifacts across multiple data sources. So analysts will observe:

- Repeated outbound connections to raw IP addresses, rather than domain names
- Persistent command and control communication, and the use of uncommon or shifting ports.
- Such traffic displays stable patterns in timing, volume, and request structure.

Definition

These are some definitions I noted down while doing my research for the Emotet section. Certain terms were recurring in many papers, sources, data, packets, etc.. I kept track of them here in case I needed any refresher while working, research, and note taking.

! **I do NOT take credit for the wording of all these definitions, I have used google and AI to take note of some of these definitions.**

Definitions:

- **Remote attack servers**: computers on the internet that hackers control. When Emotet makes it to your machine, it needs instructions; what should I steal? Should I download more malware? Do I wait? Emotet will contact the attacker's server so it has some sort of instruction on what to do. Like asking your boss what to do next, a remote attack server is the boss.
- **PCAPs**: recordings of internet traffic. Packet Capture, each 'packet' is a small piece of data sent over the network. PCAPs are needed for Emotet because if Emotet talks to attacker servers, that convo travels over the network. PCAPs capture that convo, and the PCAPs are used to see things like: Which server Emotet contacted, when it contacted it, and how often it checks in.
- **Outbound network connections**: when your computer sends data OUT to the internet. Inbound is data coming to you, and outbound is data leaving from you. With Emotet, outbound connections are made silently in the background. Example with Emotet: Your laptop sends data to: 185.xxx.xxx.xxx (attacker server), every few minutes… but you aren't doing anything. That's sus.
- **Secondary payload data**: extra malware that gets downloaded after the first infection. This is often <u>ransomware, password stealers, and network-spreading malware</u>. Recall how Emotet works:
    1. Emotet infects the computer
    2. It will connect to attacker servers
    3. Attacker says 'download this new malware'
    4. Emotet downloads and runs it. That new malware = secondary payload.
- **Registry**: like a control panel database. Most normal apps will read to it, they rarely write to it. This is significant because malware writes to it a lot in order to start on reboot and stay hidden. This is what makes the registry write frequency high, which is suspicious.
- **File path entropy:**
    - ➔ Normal software: C:\Program Files\Google\Chrome\
    - ➔ Malware: C:\Users\AppData\Roaming\xJ92kLq0\A8f9.exe

- **Beaconing**: when the infected computer contacts the same server again and again. Emotet does this to ask if there's any new instructions. This may be contacting every 5 minutes, if it's the same IP, same packet size.
- **Destination IP reputation**: if you speak to a suspicious server, like a random IP in another country, that's sketchy and suspicious. There are several known bad servers.
- **New process trees**: new chain of programs launching other programs. Office apps SHOULDN'T launch system tools.
    - ➔ Normal: Chrome → website
    - ➔ Malware: Word → Powershell → CMD → unknown.exe
- **Internal hosts** = victims, peers, infrastructure
- **External IPs** = potential C2 servers
- **C2**: command and control, the system attackers use to remotely control infected machines.